

ASSIGNMENT 03 · STARTUP BUILD

Before every launch, one scary question.

**Will anyone actually buy
this?**

LaunchLens · Ruby Gunna · LangGraph + SerpApi + Oxylabs

LAUNCH

THE PRODUCT

LaunchLens turns that fear into a **verdict.**

A market-intelligence copilot for founders. Ask in plain English, and it fuses live demand signals from Google with live supply signals from Amazon into one **Go / No-Go / Niche** call — with a price band, a positioning angle, and a confidence level.

THE PROBLEM

The signals never meet.

Demand lives on Google

Trends, News, Shopping — is anyone searching, is it seasonal, what does it cost?

Supply lives on Amazon

Who already sells it, at what price, and what do the reviews complain about?

Nobody fuses them

A trend chart says 'maybe'. A competitor list says 'crowded'. The answer is in the overlap.

WHAT IT DOES

One question in, one judgement out.

01

Ask

"Should I launch a \$20 meal-prep set in the US?"

02

Gather

Pulls demand + supply live, in parallel.

03

Judge

Go / No-Go / Niche + price band + confidence.

04

Remember

Follow-ups keep context across the chat.

DEMO FLOW

A founder conversation, end to end.

“...worth launching a 32oz steel bottle under \$40?”

full fan-out → verdict

“What are reviewers complaining about?”

mined Amazon complaints

“What about a premium glass version at \$35?”

remembers the product —
memory

“What did we conclude overall?”

summarized recall

 Built on LangGraph

Everything, wired into one graph.

Router

read the question

Fan-out

pull sources in parallel

Agent + Tools

SerpApi & Oxylabs

Memory

short-term +
summarize

UNDER THE HOOD

The five concepts, mapped to code.

1

Graph & state

Typed StateGraph + reducer channels

state.py 16/28 · graph.py 28

2

Fan-out (parallel)

Router returns a 3-node list, results merge

graph.py 49–65 · nodes.py
98/104/110

3

Routing

Intent → conditional edges, chat = default

router.py 65/81 · graph.py 49

4

Agent + tools

LLM ⇌ ToolNode loop over 6 tools

nodes.py 135 · graph.py 41/68

5

Short-term memory

Checkpointers + summarization

main.py 140 · nodes.py 64

DEMAND, MARKET AND PRICE — FUSED INTO ONE ANSWER

DEMAND

is the interest really there?

MARKET

who already wins, and why?

PRICE

where would yours land?

GO

or no-go. One clear call.

SIX TOOLS, TWO PROVIDERS

Slim JSON in, evidence out.

DEMAND · SerpApi

google_trends

interest + seasonality, slope-classified

google_news

launches, recalls, competitor moves

google_shopping

cross-retailer price band

SUPPLY · Oxylabs

amazon_search

live listings, prices, review counts

amazon_product

per-ASIN mined review complaints

amazon_bestsellers

how entrenched the competition is

Every tool returns a few fields, never the raw payload — tokens + context stay bounded.

SHORT-TERM MEMORY

Survives restarts and long chats.

Checkpointer

SqliteSaver persists full state after every node, keyed by thread_id — quit and resume.

Summarization

Old plain messages fold into a rolling summary once the chat grows.

History intact

Only Human/AI text is compressed; tool-call sequences are never broken.

Flat cost

Per-turn cost stays roughly constant instead of growing with the conversation.

VERDICT + CONFIDENCE

A call you can trust — and challenge.

Conditional, not absolute

"Go IF COGS < \$X and a defensible feature exists."

Confidence level

High / Medium / Low — lowered when a source comes back empty.

Trend sanity

Treats Trends as a relative, seasonal index — 5→10 isn't 'doubling'.

Unit-economics aware

Prompts margin reasoning: COGS, fees, ad cost before a Go.

ROADMAP · THANKS

From signal to **verdict** — in one chat.

- Next: a real FBA fee + COGS unit-economics module (margin, not just a price gap).
- Postgres checkpointer for multi-user prod · a small eval harness · richer streaming UI.
- Repo: github.com/SQLicious/LaunchLens_Product_Validator

SHIP